# Interim Report

Templating in Standard ML

Ja Ying Lew

Supervisor: Dr. Gregerly Buday

Department of Biological, Chemical and Materials Engineering

University of Sheffield

November 10, 2025

# Contents

# 1. Background to Project

Template systems separate presentation markup from application logic, a fundamental principle in software engineering. Mustache exemplifies this through its "logic-less" design philosophy, which prohibits embedding conditional statements or loops within templates. Instead, all logic resides in the view model, with templates serving purely as formatting specifications. This strict separation has led to Mustache's widespread adoption across numerous programming languages. Despite broad implementation coverage, no documented Mustache implementation exists in Standard ML. This gap is notable given Standard ML's strong static type system, comprehensive pattern matching facilities, and sophisticated module system—characteristics that could provide compile-time guarantees about template correctness and type-safe data processing. This project develops a Mustache template engine in Standard ML, investigating how the language's features apply to template processing. The implementation will parse Mustache syntax, process data contexts, and generate output. Beyond creating a functional tool, the project evaluates Standard ML's suitability for this domain, examining both advantages (type safety, pattern matching) and limitations (library ecosystem, tooling) encountered during development.

# 2. Introduction, Aims and Objectives

** not too sure on this section ** but left short (cus i am worried for words) and similar to what i wrote in the aims and objectives which i submitted

To implement Mustache templating in Standard ML, demonstrating how functional programming and strong typing can improve template processing efficiency, reliability, and maintainability for systematic artifact generation from structured data. Secondary Aims: Show how Standard ML's strong type system, pattern matching, and functional programming paradigm provide advantages over traditional imperative approaches for template processing and artifact generation.

# 3. Literature Review and Work Progress to Date

## 3.1. Introduction

This literature review examines template systems in software engineering, focusing on the Mustache templating language and implementations in functional programming languages. Template systems address the challenge of separating presentation logic from application code, a principle known as separation of concerns. This review surveys Mustache's "logic-less" design philosophy, examines implementations in functional languages, and identifies gaps in current literature, particularly the absence of Standard ML implementations.

## 3.2. Template Systems: Purpose and Design

Template systems emerged to generate dynamic content while maintaining separation between presentation and business logic. Traditional web development often intermingled application code with markup, causing maintenance difficulties and security vulnerabilities [1]. Modern template engines provide specialised syntax for presentation structures that reference application-provided data.

Template system design involves tension between expressiveness and simplicity. Powerful template languages allow complex logic directly in templates, potentially duplicating business logic and violating separation of concerns. Conversely, restrictive systems may force awkward workarounds when presentation requires dynamic behavior [2]. Different engines resolve this tension differently, reflecting varying philosophies about boundaries between presentation and logic.

## 3.3. The Mustache Approach

Mustache represents an extreme "logic-less" philosophy, deliberately excluding conditional statements, loops, and control structures from templates [3]. Templates consist solely of tags referencing values in provided data contexts, with all logic residing in the

view model layer. This enforces strict architectural boundaries: templates serve purely as presentation specifications while controllers handle data transformation and conditional logic [2].

This approach yields several benefits: templates remain uncluttered and readable, forced separation improves maintainability by preventing business logic accumulation in presentation layers, and templates become implementation-agnostic, facilitating engine replacement [2]. However, constraints exist—developers must prepare all data variations in the model, even for simple presentational decisions, and Mustache lacks features like filters found in more expressive languages. Despite limitations, Mustache has achieved widespread adoption across programming languages [4].

## 3.4. Functional Language Implementations

Functional programming languages bring unique characteristics to template systems, particularly type safety and compile-time verification. Unlike dynamically-typed implementations, functional languages leverage sophisticated type systems to catch errors before runtime.

Haskell's Hamlet, part of the Yesod web framework, uses Template Haskell—a metaprogramming facility compiling templates into Haskell code at compile time [5]. This enables type checking of interpolated variables, catching runtime errors during compilation. Hamlet automatically prevents cross-site scripting by escaping user content and validates internal links at compile time.

Alternatively, Blaze-html uses a combinator library approach, representing HTML elements as Haskell functions . Developers compose structures using function calls rather than parsing template syntax, providing maximum type safety—invalid HTML cannot be expressed—though at the cost of unfamiliar syntax.

OCaml's Jingoo provides compatibility with Python's Jinja2 syntax. Unlike Haskell's compile-time approaches, Jingoo operates at runtime, trading type safety for syntactic familiarity, reflecting OCaml's balance between functional purity and imperative programming.

## 3.5. Gap in Literature

This review reveals a significant gap: no template system implementations exist in Standard ML literature. While Haskell and OCaml have received attention, Standard ML—despite its influential module system and strong type safety—lacks documented template engines, representing an opportunity for investigation.

# 4. Current Status of the Work

1. worked on gilmore standard ML exercises (started familarising with standard ML). 2. focused on chapters 9 and 10 and learnt about signatures, structures and functors 3. Prioritising interim report

# 5. Self-Review

Coming from a Biomedical Engineering background, I initially underestimated the foundational knowledge gap between disciplines. The Computer Science terminology alone—terms that would be second nature to CS students—presented a significant barrier. Early progress was slower than expected as I encountered concepts like signatures, primary and secondary compuation, and standard ML, each requiring substantial background reading to understand properly.

Rather than attempting to rush through unfamiliar material, I adopted a deliberate, bottom-up approach: thoroughly understanding each new term and concept before moving forward. While frustrating at times, this strategy has proven worthwhile. Individual pieces that initially seemed disconnected are now forming a coherent picture, and I am beginning to appreciate the elegance of functional programming's approach to problems I had previously only encountered in imperative contexts

# 6. Project Management

Write your project management plan here.

## Programme of Work

- Task 1 – Description
- Task 2 – Description
- Task 3 – Description
- ...

# A. Gantt Chart

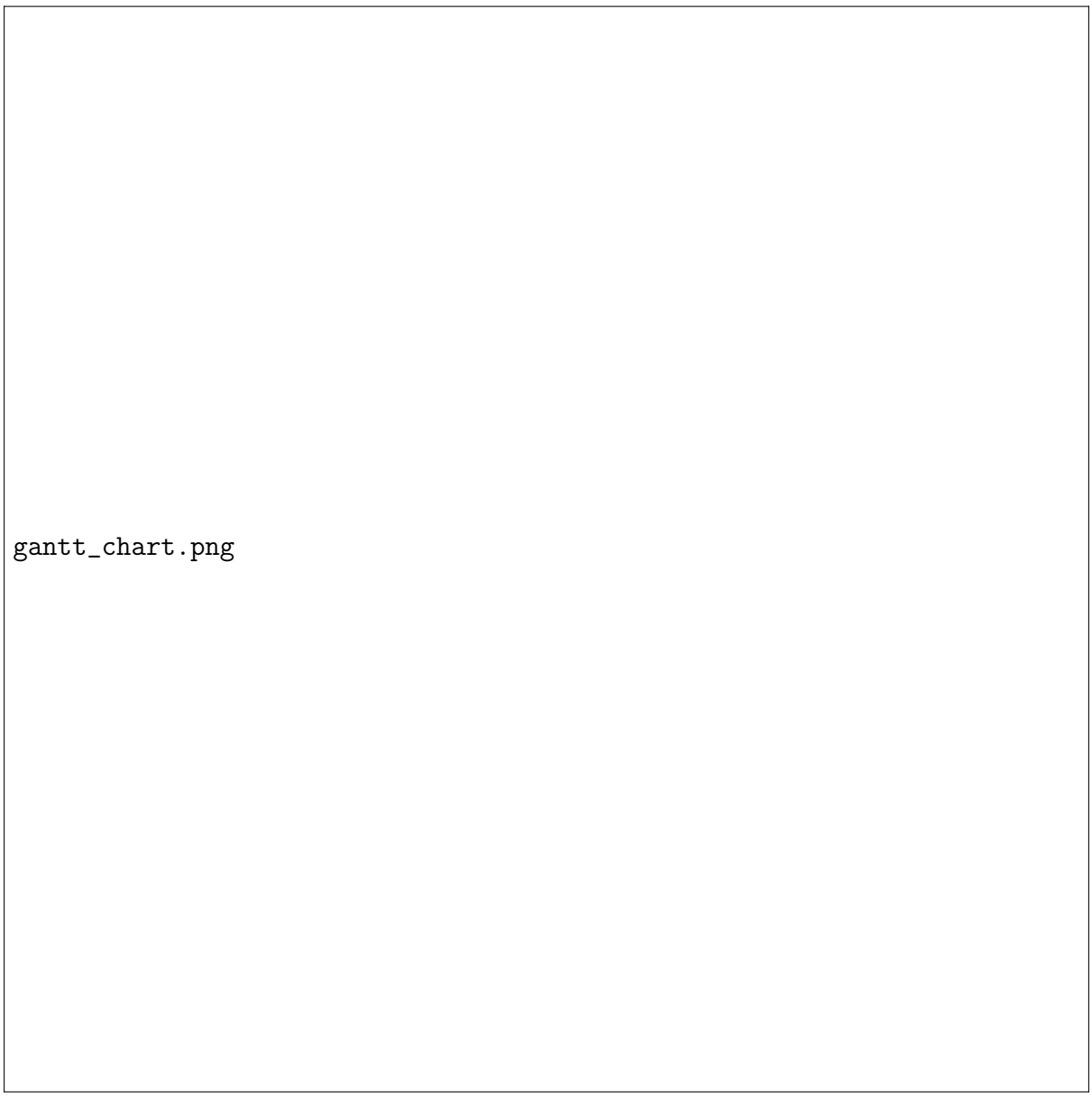


Figure A.1: Updated Gantt Chart showing progress to date.